

词法分析：导论与正则表达式

Lecture 1 课件总结

编译原理 - 中山大学计算机学院 - 赵帅

根据课件 1.Lexical Analysis_Intro_RE.pdf 整理

1 整体脉络

词法分析是编译器前端的第一步。它从源程序的字符流出发，将字符切分成一串 token，再交给语法分析器构建语法树。课件的核心路线是：

字符流 → 词法分析器 → token stream → 语法分析器 → syntax tree

本讲关注两个问题。第一，什么是 token、lexeme、token class，以及词法分析器如何扫描输入。第二，如何用正则表达式描述 token 的模式，为后续从正则表达式构造自动机做准备。

2 词法分析的任务

2.1 基本定义

- **Lexical Analysis**: 识别源程序中的子串，也就是 lexeme，并根据模式将其归类，生成 token。
- **Token**: 语言中具有意义的最小单位，通常写成二元组 $(class, lexeme)$ 。其中 class 是抽象类别，lexeme 是源代码中的实际字符序列。
- **Lexeme**: 源程序中匹配某个 token 模式的一段字符，是某个 token 的实例。
- **Token class**: token 的类别，例如 keyword、identifier、number、operator、whitespace 等。

例如代码片段

```
while (i < z) i++;
```

可以被划分为 keyword while、左括号、identifier i、关系运算符 <、identifier z、右括号、identifier i、自增运算符 ++ 和分号等 token。

2.2 词法分析器做什么

一个词法分析器通常完成四类工作：

1. 读取源程序字符流。
2. 删除或跳过注释、空白符、换行、制表符等对语法结构无直接意义的内容。
3. 根据模式切分输入，识别每个 lexeme 的 token class。
4. 将 token stream 提供给 parser；parser 依赖 token class 而不是原始字符来判断语法角色。

以 `a=3.14` 为例，词法分析器可以输出

`<id, a>`, `<op, =>`, `<number, 3.14>`.

3 词法分析中的设计问题

3.1 token class 的设计

词法分析首先要定义有限集合的 token class。这些类别取决于语言本身，也取决于 parser 的设计。例如常见类别包括：

- **number**: 一个或多个连续数字，或更复杂的整数、浮点数格式。
- **keyword**: 语言保留字，如 `if`、`else`、`for`、`while`。
- **whitespace**: 空格、制表符、换行等。
- **identifier**: 用户自定义名称，通常用于变量、函数、类型等实体。

设计上的关键是判断“哪些字符串属于哪个类别”。例如 `if` 既可能满足 `identifier` 的模式，也可能是 `keyword`；`==` 不能被错误切成两个 `=`。

3.2 歧义、最长匹配和向前看

课件强调词法分析中可能出现歧义。典型例子包括：

- `if` 可以是 `keyword`，也满足 `identifier` 的模式。
- `==` 可以看成是一个相等运算符，也可以错误地看成两个赋值符号。
- C++ 中的 `vector<vector<int>>` 里，`>>` 可能被理解为流运算符，也可能是两个模板右尖括号。

常用消歧规则有：

1. **关键字优先**: 当一个字符串既可以是 `keyword` 又可以是 `identifier` 时，优先识别为 `keyword`。
2. **最长匹配**: 每次尽可能匹配最长 token。例如 `==` 优先于单个 `=`。
3. **规则顺序**: 若仍然冲突，优先采用词法规范中列在前面的规则。

有些语言结构需要 look ahead，甚至需要 parser 反馈才能完全消歧。这会增加词法分析设计复杂度。因此通常希望 token 之间尽量不重叠，使扫描器可以从左到右一趟完成。

4 形式语言基础

4.1 字母表、串和语言

正则表达式要描述的是语言。课件给出三个基础概念：

- **Alphabet** Σ : 有限符号集合，例如 $\{0, 1\}$ 、 $\{a, b, c\}$ 或 ASCII 字符集。

- **String**: 字母表中符号组成的有限序列。空串记为 ϵ ，长度为 0。
- **Language**: 固定字母表上的任意可数串集合。

需要区分空语言 $\emptyset$ 和只包含空串的语言 $\{\epsilon\}$ 。前者没有任何字符串，后者有一个字符串，只是该字符串长度为 0。

4.2 语言运算

课件介绍了三种用于构造语言的基本运算：

- **并**: $A \cup B$ ，正则表达式中常写作 $A | B$ 。
- **连接**: AB ，表示从 A 中取一个串、从 B 中取一个串后拼接所得的所有结果。
- **Kleene 闭包**: L^* ，表示将 L 中的串连接零次或多次得到的所有结果，包含 ϵ 。

正闭包 L^+ 表示连接一次或多次：

$$L^+ = L^1 \cup L^2 \cup L^3 \cup \dots, \quad L^* = \{\epsilon\} \cup L^+.$$

例如 $L = \{a, b\}$ ，则

$$L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, \dots\}.$$

5 正则表达式

5.1 正则表达式的作用

正则表达式是描述字符串模式的简单记法。它不够表达完整的自然语言或完整的 C 语言语法，但非常适合描述编程语言中的 token，例如 identifier、integer、keyword 和 whitespace。

能够被正则表达式描述的语言称为正则语言。词法分析的典型 workflow 是：

用正则表达式指定 token pattern $\rightarrow$ 构造 NFA/DFA $\rightarrow$ 实现 token recognizer $\rightarrow$ 生成词法分析器

5.2 递归构造

正则表达式由更小的正则表达式递归构成。每个正则表达式 r 表示一个语言 $L(r)$ 。

形式	含义
ϵ	匹配空串, $L(\epsilon) = \{\epsilon\}$
a	对任意符号 a ，只匹配字符串 a , $L(a) = \{a\}$
(r)	与 r 表示同一语言
$(r) (s)$	表示 $L(r) \cup L(s)$
$(r)(s)$	表示 $L(r)L(s)$ ，即连接
$(r)^*$	表示 $(L(r))^*$ ，即零次或多次重复

注意空集 $\emptyset$ 不是空串 ϵ 。 $\emptyset$ 对应没有字符串， ϵ 是一个长度为 0 的字符串。

5.3 运算符优先级

正则表达式的运算符优先级从高到低是：

括号 (A) > 闭包 A^* > 连接 AB > 并 $A | B$.

因此 $ab*c | d$ 的解析方式是

$$((a(b^*))c) | d.$$

6 常用正则表达式写法

6.1 常见缩写

写法	含义
A^+	一个或多个 A , 等价于 AA^*
$A?$	零个或一个 A , 等价于 $A \epsilon$
$[a-z]$	字符范围, 等价于 $a b \dots z$
$[^a-z]$	排除范围, 表示不在 $a-z$ 中的字符
$^$ 和 $$$	分别匹配行首和行尾, 具体含义由上下文决定

identifier 的典型定义为：

$$\begin{aligned} letter &= [A-Za-z] \\ digit &= [0-9] \\ identifier &= letter(letter | digit)^* \end{aligned}$$

6.2 编程语言中的常用记号

记号	含义
$\backslash d$	任意十进制数字, 即 $[0-9]$
$\backslash D$	任意非数字字符, 即 $[\sim 0-9]$
$\backslash s$	任意空白字符, 如 tab、newline、return 等
$\backslash S$	任意非空白字符
$\backslash w$	字母、数字或下划线, 即 $[a-zA-Z0-9_]$
$\backslash W$	非字母、非数字、非下划线字符
$\cdot$	任意字符; 若要匹配点号本身, 需要写成 $\backslash \cdot$
$(\dots)$	捕获匹配内容

7 从正则表达式到词法规范

课件给出一个规范化步骤：

1. 为每个 token class 写一个正则表达式。例如 $Numbers = digit^+$, $Keywords = if | else | \dots$, $Identifiers = letter(letter | digit)^*$ 。
2. 构造一个能匹配所有 token lexeme 的总表达式：

$$R = R_1 | R_2 | R_3 | \dots$$

3. 对输入 $x_1x_2\cdots x_n$ ，从左向右检查某个前缀 $x_1\cdots x_i$ 是否属于 $L(R)$ 。
4. 若匹配成功，则该前缀属于某个 $L(R_j)$ ，输出相应 token。
5. 从输入中删除该前缀，继续扫描；若没有规则匹配，则报告词法错误。

这个过程揭示了正则表达式的定位：正则表达式只是语言规范，真正执行匹配还需要实现机制。下一步通常是把正则表达式转换成有限自动机。

8 典型例子与结论

8.1 正则表达式例子

正则表达式	表示的语言
a^*	零个或多个 a ，包含 ϵ
a^+	一个或多个 a
$(a b)(a b)$	$\{aa, ab, ba, bb\}$
$(a b)^*$	所有由 a 和 b 组成的字符串，包含 ϵ
$(aa ab ba bb)^*$	所有由 a 、 b 组成且长度为偶数的字符串
$0([0-9])^*0$	以 0 开始且以 0 结束的数字串
$1^*(0 \epsilon)1^*$	至多包含一个 0 的二进制串
$(0 1)^*00(0 1)^*$	包含子串 00 的二进制串

不同正则表达式可能表示同一个语言。例如 $(a|b)^*$ 与 $(a^*b^*)^*$ 都能表示所有由 a 和 b 构成的字符串。

8.2 练习要点

- $\text{if } i == 0$ 的 token 序列可以写为 $\langle \text{keyword}, \text{if} \rangle$ 、 $\langle \text{id}, i \rangle$ 、 $\langle \text{op}, == \rangle$ 、 $\langle \text{num}, 0 \rangle$ 。
- 正则表达式用于指定 token pattern；有限自动机用于实现 token recognizer。
- $(x|y)(x|y)$ 表示 $\{xx, xy, yx, yy\}$ 。
- $(0|1)^*0(0|1)^*0(0|1)^*$ 表示至少包含两个 0 的二进制串。
- 所有包含子串 aba 的 a/b 串可写为 $(a|b)^*aba(a|b)^*$ 。
- 所有偶数长度的 a/b 串可写为 $(aa|ab|ba|bb)^*$ 。
- 所有不包含子串 abb 的 a/b 串可写为 $b^*(a|ab)^*$ 。

9 一页复习版

- 词法分析把字符流切分为 token stream，是 parser 的输入。
- token 是 $(\text{class}, \text{lexeme})$ ；lexeme 是源代码中的实际字符序列。
- token class 由语言和 parser 需求共同决定。
- 词法分析器通常跳过注释和空白符。
- 歧义通过关键字优先、最长匹配和规则顺序处理。

- 字母表是有限符号集；串是有限符号序列；语言是某字母表上的串集合。
- 正则表达式通过并、连接、闭包描述正则语言。
- 正则表达式适合描述 token，但不是词法分析器本身；实现通常依赖 NFA/DFA。
- 下一讲的自然问题是：如何从正则表达式自动构造可执行的 token recognizer。